

Rovel: AWS EC2 Production Deployment Guide

This guide provides a sequential, step-by-step walkthrough to deploy the Rovel PaaS platform on an AWS EC2 instance. Each step is documented with the commands used, their meaning, and the exact permissions and configuration adjustments needed beforehand to prevent errors.

Core Architecture Overview

Rovel is a self-hosted Platform-as-a-Service (PaaS). It runs a Next.js web dashboard (control plane) and a Node.js worker daemon. The worker compiles and deploys developer applications in isolated Docker containers, dynamically routing traffic to them via Nginx reverse proxies and securing them with Let's Encrypt SSL certificates.

Step 1: AWS EC2 Instance Provisioning

In your AWS EC2 Console, configure the following settings to launch your virtual server:

- Name: rovel-server
- Operating System (AMI): Select Ubuntu Server 24.04 LTS (HVM).
- *Reason*: The automated bootstrap script (setup.sh) is designed for Debian/Ubuntu environments (apt-get). It will fail on Amazon Linux, RHEL, or SUSE.
- Instance Type: Select t3.small (2 vCPUs, 2 GiB RAM).
- *Reason*: Compiling Next.js, React, or Python applications during container builds consumes significant memory. While t3.micro (1 GB RAM) can be used with large swap files, t3.small offers a much more stable and faster build experience.
- Key Pair: Create or select an RSA key pair in .pem format (e.g., rovel-key.pem).
- Storage: Set the root volume size to 30 GiB (type gp3).
- *Reason*: Docker image layers and build caches occupy substantial disk space. AWS provides up to 30 GB of SSD storage for free under the 12-Month Free Tier.

Network Security Groups

Configure your security group rules to allow the following public ingress traffic:

Port	Protocol	Source	Purpose
22	TCP (SSH)	Anywhere (`0.0.0.0/0` or My IP)	Remote terminal access
80	TCP (HTTP)	Anywhere (`0.0.0.0/0`)	Domain validation and HTTP routing
443	TCP (HTTPS)	Anywhere (`0.0.0.0/0`)	Secure SSL application routing

Step 2: DNS & Domain Configuration

Go to your DNS registrar (e.g., Name.com, Route 53) and point your domain records to the Public IPv4 address of your EC2 instance (e.g., 13.204.53.200). Create the following three A records:

Type	Host / Name	Answer / Value	Purpose
A	`rovel.dev` (or `@`)	`YOUR_EC2_PUBLIC_IP`	Main landing website
A	`console`	`YOUR_EC2_PUBLIC_IP`	Developer dashboard console

****A******`\*.apps`****`YOUR\_EC2\_PUBLIC\_IP`****Wildcard routing for deployed developer app**

Step 3: Server Connection & Key Permissions

1. Restrict Key Permissions (Windows PowerShell)

Windows SSH client requires that your private key file (.pem) is only accessible by your current user account. Navigate to the folder containing your key (usually ~\Downloads) and run:

```
icaccls.exe rovel-key.pem /grant:r "$($env:USERNAME):(R)"
icaccls.exe rovel-key.pem /inheritance:r
```

***Reason*:** If key permissions are too open, the SSH connection will be rejected with an "Unprotected Private Key File" error.

2. Connect via SSH

```
ssh -i rovel-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

Step 4: Configure Virtual Memory (Swap Space)

Even on t3.small (2 GB RAM), container builds can spike memory usage and trigger the Out-Of-Memory (OOM) killer. Set up a 4 GB swap file on your SSD:

```
# Create a 4 GB blank file
sudo fallocate -l 4G /swapfile
# Restrict file access to root
sudo chmod 600 /swapfile
# Format it as swap space
sudo mkswap /swapfile
# Enable the swap space
sudo swapon /swapfile
# Persist across system reboots
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Verify with `free -h`. You should see Swap: 4.0Gi.

Step 5: Install Docker CE (Official)

Install Docker CE using the official installation script:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER
newgrp docker
```

***Reason*:** The default Ubuntu package manager (apt) does not contain the official docker-ce or docker-compose-plugin packages in its default repositories.

Step 6: Install Node.js, Nginx, and System Packages

Install the remaining dependencies and configure system variables:

```
# Install Node.js v22 and npm
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs nginx certbot python3-certbot-nginx

# Configure Nginx passwordless reload for the worker
echo "$USER ALL=(ALL) NOPASSWD: /usr/sbin/nginx" | sudo tee /etc/sudoers.d/rovel-nginx

# Create builds directory and adjust permissions
sudo mkdir -p /opt/rovel/builds
sudo chown -R $USER:$USER /opt/rovel

# Open firewall rules
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
```

Step 7: Apply Pre-Emptive Directory Permissions

Before running the application, you must adjust folder permissions so that the worker daemon (running as the ubuntu user) can write Nginx routing configs and read SSL keys without encountering permission errors:

```
# 1. Grant the 'ubuntu' user permission to write Nginx configurations
# (Prevents EACCES: permission denied errors during deployments)
sudo chown -R ubuntu:ubuntu /etc/nginx/sites-enabled/

# 2. Grant the 'ubuntu' user permission to read the SSL certificates
# (Allows the worker to traverse letsencrypt directories and read private keys)
sudo chmod 755 /etc/letsencrypt/
sudo chmod -R 755 /etc/letsencrypt/live/
sudo chmod -R 755 /etc/letsencrypt/archive/
```

Step 8: Configure Environment Variables

Create and configure your production environment file in the root ~/Rovel folder:

```
nano .env.production
```

Paste your production configuration. Ensure the following values are configured:

```
# Database Credentials
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/rovel?schema=public"
REDIS_URL="redis://localhost:6379"
DB_USER=postgres
DB_PASSWORD=YOUR_PASSWORD
DB_NAME=rovel

# Domain Configuration
BASE_DOMAIN="apps.rovel.dev"
NEXT_PUBLIC_BASE_DOMAIN="apps.rovel.dev"

# Security (32-byte hex keys generated using openssl)
JWT_SECRET="YOUR_JWT_SECRET"
```

```
ENCRYPTION_KEY="YOUR_32_CHAR_ENCRYPTION_KEY"
```

```
# GitHub OAuth App (Select the app matching the console.rovel.dev domain)
GITHUB_CLIENT_ID="0v23lipY6oaKaX3j80hg"
GITHUB_CLIENT_SECRET="YOUR_GITHUB_OAUTH_SECRET"
```

***Note*:** Ensure that the GITHUB_CLIENT_ID and GITHUB_CLIENT_SECRET match your registered GitHub Developer Settings for Rovel (Homepage: <https://console.rovel.dev>, Callback: <https://console.rovel.dev/api/auth/callback>).

Step 9: Boot Services and Compile Code

Run these commands to start the databases, install packages, sync schemas, and compile the platform:

```
# 1. Start Postgres and Redis containers
docker compose up -d

# 2. Install application dependencies
npm install

# 3. Push database schemas
node -e "const fs = require('fs'); const path = require('path'); const envPath = fs.existsSync('.env.production') ? '.env.production' : '.env'; require('dotenv').config({ path: envPath }); require('child_process').spawn('npm', ['run', 'db:push', '-w', 'packages/db'], { stdio: 'inherit', shell: true })"

# 4. Compile Next.js and worker code
# (Run this command to build the spaces. If dependency order issues arise, running it a second time completes the build)
npm run build:all
```

Step 10: Configure Nginx Routing

Copy the server block configuration and remove the default index page:

```
sudo cp infrastructure/nginx/rovel.conf /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
```

Open /etc/nginx/sites-enabled/rovel.conf:

```
nano /etc/nginx/sites-enabled/rovel.conf
```

Verify that the server_name directive includes both the console and root domain so they both resolve to Next.js on port 3000:

```
server_name console.rovel.dev rovel.dev;
```

Reload Nginx:

```
sudo systemctl reload nginx
```

Step 11: Generate SSL Certificates

1. Provision SSL for the Dashboard and Root Domain

```
sudo certbot --nginx -d console.rovel.dev -d rovel.dev
```

Note: When Certbot prompts you, select Expand (E) to bundle both domains into the same certificate.

2. Generate the Wildcard SSL Certificate (For User Applications)

Request a wildcard SSL certificate via a manual DNS TXT challenge:

```
sudo certbot certonly --manual --preferred-challenges=dns -d "*.apps.rovel.dev" -d "apps.rovel.dev"
```

1. Certbot will output a verification value.
2. Go to your DNS registrar (Name.com) and add a TXT record:
 - Host: _acme-challenge.apps
 - Answer: PASTE_THE_CERTBOT_VALUE
3. Wait 60 seconds for DNS propagation, then press Enter in your SSH terminal.

Step 12: Start Services in PM2

Install PM2 and start the background applications:

```
# Install PM2 globally
sudo npm install -g pm2

# Start the dashboard and worker
pm2 start npm --name "rovel-dashboard" -- run start -w apps/web
pm2 start dist/index.js --name "rovel-worker" --cwd apps/worker

# Configure PM2 to restart automatically on system reboots
pm2 startup systemd
# (Copy and execute the command printed in the terminal output of the step above)
pm2 save
```

Note on Client-Side Rebuilds

Environment variables starting with NEXT_PUBLIC_ are baked statically into the Next.js bundle during the build phase. If you modify .env.production later, you must re-run npm run build:all and pm2 restart all --update-env to apply the changes to the browser dashboard.